

RL TRADING

EXPERIMENTS, PAPERTRADING
REINFORCED LEARNING



Reinforcement Learning Projekt

Von Haris Salii und Fenlin Chirakal

1. Summary

Das vorliegende Dokument beschreibt die Analyse und experimentelle Methodik des Projekts. Primäres Ziel war die Entwicklung eines autonomen Handelsagenten, der mittels Deep Reinforcement Learning (DQN) profitable Entscheidungen unter Berücksichtigung realer Marktdaten trifft. Der Bericht dokumentiert den strategischen Übergang von einem einfachen Demonstrationskonzept hin zu einem spezialisierten, simulationsgetreuen Forschungswerkzeug, das auch komplexe Marktfriktionen wie Gebühren erfolgreich bewältigt. Die hier aufgeführten Inhalte dienen als zusammenfassende Darstellung der erzielten Ergebnisse; für eine detaillierte Einsicht in die Quellcodes, Datensätze und technischen Logs verweist dieses Dokument auf das zugehörige GitHub-Repository. (<https://github.com/Haris-1-1/Reinforcement-Learning-Trading-Bot-RLTrader->)

Inhalt

1. Summary	2
2. Einleitung und Problemstellung	4
2.1. Ziel	4
2.2 Vorgehensweise/Methodik.....	4
4. Phase II: Aufbau der Simulation und Agenten-Selektion	5
4.1. Advanced Trading Environment (A.T.E.).....	5
4.1.1 Zustandserweiterung und Feature-Engineering	5
4.1.2 Analyse der Technischen Indikatoren.....	5
4.1.3 Die Constraints-Schicht (Marktfriktionen).....	6
4.2. Agenten-Architekturen im Vergleich	6
4.2.1 Die Baseline: Tabellarisches Q-Learning	6
4.2.2 Der Herausforderer: Deep Q-Network (DQN)	7
4.2.3 Proximal Policy Optimization (PPOAgent).....	7
5. Experimente, Training und Validierung	8
6. Strategische Neuausrichtung Der Enhanced "Smart Money" DQN.....	9
6.1 Motivation und Problemstellung	9
6.2 Kern-Architektur: Windowed Deep Q-Learning.....	9
6.2.1 Sequenz-Logik (Observation Windowing)	9
6.3 Feature Engineering & Smart Money Indikatoren	9
6.3.1 Hybrid-Komponente (Supervised "Intuition")	10
6.3.2 Whale-Tracking & Marktstruktur	10

6.3.3 Volatilität & Risikomanagement	10
6.3.4 Cyclical Time Encoding	10
6.4 Efficiency Boosters (Lern-Optimierung).....	10
6.5 Validierung & Benchmarks.....	11
6.6 Grenzen des Konzepts & Evolution zum finalen System	11
7. Implementation des Finalen Systems : Dueling Double DQN.....	12
7.1 Die Architektur: Dueling Double DQN.....	12
7.2 Das erweiterte "Auge" des Agenten: 42 Features	12
7.3 Die Umgebung: Gymnasium & Action Masking.....	13
7.4 Benchmarking und Validierungsergebnisse	13
8. Zusammenfassung der Ergebnisse.....	15
8.1 Schlussfolgerungen	15
8.2 Kritische Reflexion und Ausblick	16
Hilfsmittelverzeichnis/ Tools	18
Quellenverzeichnis.....	18

2. Einleitung und Problemstellung

2.1. Ziel

Das zentrale Ziel des Projekts ist die Durchführung eines Forschungsexperiments zur systematischen Evaluierung der Rentabilität und Qualität verschiedener Reinforcement Learning (RL)-Strategien. Hierfür dient ein Paper-Trading-Bot als experimentelle Testumgebung. Der Fokus liegt auf der Entwicklung eines Agenten, der optimale Handelsentscheidungen autonom erlernt, indem er – analog zum menschlichen "Trial-and-Error"-Prinzip – seine Strategie basierend auf direktem Marktfeedback (Gewinn/Verlust) kontinuierlich adaptiert.

2.2 Vorgehensweise/Methodik

1. Phase I: Initialer Proof-of-Concept (Demo)

Um die fundamentalen Prinzipien des Reinforcement Learning im Kontext des Handels zu explorieren, wird zunächst ein vereinfachter Prototyp in Form eines Demo-Bots entwickelt. Die geplante Umsetzung erfolgt über ein fokussiertes Skript (`live_paper_trading.py`), welches auf einer grundlegenden Umgebung mit einem reduzierten Aktionsraum basiert.

Die primäre Zielsetzung liegt hierbei explizit nicht auf der Erzielung maximaler Profitabilität, sondern auf dem Aufbau eines tiefgreifenden technischen Verständnisses. Im Zentrum steht die Untersuchung, wie ein Agent Marktdaten wahrnimmt, wie Entscheidungsprozesse initiiert werden und welche konkreten Auswirkungen einzelne Aktionen – wie Kaufen oder Verkaufen – auf den Portfolio-Status haben. Diese Phase fungiert somit als essenzielles technisches Fundament für die darauf auf folgenden, komplexeren Entwicklungsstufen

2. Phase II: Aufbau der Simulation und Agenten-Selektion

Im zweiten Schritt wird eine professionelle, realitätsnahe Simulationsumgebung (`AdvancedTradingEnv`) geschaffen, die Marktfriktionen wie Gebühren (0,1 %) und Slippage berücksichtigt. Auf dieser Basis sollen verschiedene Algorithmen gegeneinander getestet werden, um den geeignetsten Kandidaten zu identifizieren:

Q-Learning: Wird als Baseline implementiert, um zu prüfen, ob einfache Verfahren ausreichen.

DQN (Deep Q-Network): Wird evaluiert, um festzustellen, ob neuronale Netze komplexe Marktzustände besser verarbeiten können.

PPO: Wird als Alternative für stochastische Policies getestet.

3. Phase III: Iterative Optimierung und Leistungssteigerung

Nach der Selektion des am besten geeigneten Basis-Agenten wird dieser in einer intensiven Optimierungsphase umfassend weiterentwickelt, um eine stabile

Profitabilität auch unter schwierigen Marktbedingungen zu gewährleisten. Ein zentraler Baustein ist dabei das erweiterte Feature-Engineering: Hierbei wird systematisch untersucht, welche zusätzlichen Marktdaten über die reinen Preisbewegungen hinaus – wie etwa Volumen-Divergenzen oder institutionelle Order-Flow-Signale – dem Agenten helfen können, die zugrunde liegende Marktstruktur besser zu interpretieren.

Parallel dazu werden architektonische Verbesserungen implementiert, um den Agenten durch fortgeschrittene Techniken und gezielte Anpassungen am neuronalen Netzwerk oder der Lernstrategie (z. B. Dueling-Architekturen) robuster gegenüber Marktrauschen zu machen. Das übergeordnete Ziel dieser Phase ist die kontinuierliche Verfeinerung des Gesamtsystems, bis der Agent in der Lage ist, die anfallenden Transaktionskosten konsistent zu überwinden und ein positives Alpha zu generieren.

4. Phase II: Aufbau der Simulation und Agenten-Selektion

In dieser Phase wurde die Demo in eine operationale Testumgebung überführt. Der Fokus lag auf der Schaffung realistischer Marktbedingungen und dem systematischen Vergleich verschiedener Lernalgorithmen.

4.1. Advanced Trading Environment (A.T.E.)

Die Simulationsumgebung bildet das Fundament für valides Training. Sie wurde gegenüber dem Demo-Status massiv erweitert, um reale Marktmechanismen abzubilden.

4.1.1 Zustandserweiterung und Feature-Engineering

Die Basis des Zustandsraums bilden die rohen OHLCV-Preisdaten, welche durch ein umfangreiches Set an technischen Indikatoren ergänzt wurden. Eine strukturelle Besonderheit der Implementierung stellt dabei der duale Datenstrom dar, der eine strikte Trennung der Informationsverarbeitung vornimmt. Hierbei werden alle Features zunächst mittels Z-Score-Normalisierung in einen normalisierten Vektor überführt – ein kritischer Schritt, um den Gradientenabstieg bei der Nutzung tiefer neuronaler Netze (wie DQN oder PPO) zu stabilisieren. Parallel dazu bleiben die originalen, unskalierten Schlusskurse im Hintergrund erhalten und werden exklusiv für die exakte Berechnung von Gewinnen und Verlusten (PnL), Gebühren sowie des Portfoliowerts herangezogen. Diese technische Entkopplung gewährleistet, dass der Agent auf statistisch optimierten Daten lernt, während die finanzielle Abrechnung präzise und unverfälscht in USD erfolgt.

4.1.2 Analyse der Technischen Indikatoren

Die gewählten Indikatoren decken die komplette Marktdynamik ab:

- **Trend & Momentum:** Moving Averages (MA5, MA20, MA50) liefern den Kontext über verschiedene Zeithorizonte. Der MACD dient als Signalgeber für

Trendwechsel, während der RSI (Relative Strength Index) überkaufte oder überverkaufte Zustände identifiziert.

- **Volatilität & Risiko:** Bollinger Bands visualisieren die relative Preislage ("Dehnung"), während die ATR (Average True Range) die absolute Volatilität misst. Letztere zwingt den Agenten implizit, die Markturruhe in seine Risikobewertung einzubeziehen.

4.1.3 Die Constraints-Schicht (Marktfriktionen)

Constraint-Typ	Implementierung (Standard)	Finanzieller Einfluss	Agenten-Ziel
Trading Fees	Maker (0.1%), Taker (0.2%)	Direkter Kostenabzug	Vermeidung von HFT mit geringer Marge
Slippage	0.1% Preisimpakt	Reduzierter Realisierungspreis	Berücksichtigung der Liquidität
Frequency Penalty	Negative Belohnung (z.B. -0.00005)	Indirekte Bestrafung	Förderung längerer Haltezeiten ("Sniper")
Execution Delay	Konfigurierbare Latenzschritte	Asynchrone Ausführung	Robustheit gegen Latenz

4.2. Agenten-Architekturen im Vergleich

Die modulare Struktur erlaubte den direkten Vergleich verschiedener RL-Klassen, um die effektivste Methode für die Phase III zu identifizieren.

4.2.1 Die Baseline: Tabellarisches Q-Learning

Der Q-Learning Agent fungierte als fundamentale Baseline und Referenzpunkt für die Studie. Er operiert analog zu einem erfahrungsbasierten Händler, der historische Erfolgswahrscheinlichkeiten speichert, um zu bewerten, welche Aktion in einer spezifischen Marktsituation den höchsten Ertrag verspricht. Da klassisches Q-Learning jedoch nicht nativ mit dem kontinuierlichen Zustandsraum von Finanzdaten (wie beispielsweise einem exakten RSI-Wert von 62,53) umgehen kann, war eine methodische Vereinfachung mittels Diskretisierung ("Binning") erforderlich. Hierbei wurden die kontinuierlichen Indikatoren in diskrete Intervalle im Versuch beispielsweise 15 Stufen segmentiert. Das "Gedächtnis" des Agenten manifestiert sich in der Q-Tabelle, welche für jede dieser diskreten Zustandskombinationen den erwarteten Gewinnwert (Q-Wert) vorhält. Dieser architektonische Ansatz stieß jedoch bei der Erweiterung der Marktfeatures schnell an seine Grenzen: Der sogenannte "Fluch der Dimensionalität" führte bei zunehmender Anzahl von Indikatoren zu einer exponentiellen Vergrößerung der Tabelle, was ein effizientes Lernen und die Konvergenz der Strategie in komplexen Umgebungen faktisch unmöglich machte.

4.2.2 Der Herausforderer: Deep Q-Network (DQN)

Das DQN-Training (`train_dqn_agent.py`) überwindet die systemimmanenten Limitationen tabellarischer Ansätze durch die Implementierung tiefer neuronaler Netze. Das Setup umfasst die Konfiguration einer mehrschichtigen Architektur (beispielsweise [128, 128, 64] Neuronen), welche in der Lage ist, kontinuierliche und normalisierte Marktdaten direkt zu verarbeiten, ohne Informationsverluste durch vorheriges Binning in Kauf nehmen zu müssen. Zur Gewährleistung der Konvergenz in der hochvolatilen Finanzumgebung kommen zwei essenzielle Stabilisierungsmechanismen zum Einsatz: Zum einen das Experience Replay, welches vergangene Erfahrungen speichert und zufällig wiederabspielt, um schädliche zeitliche Korrelationen in den Trainingsdaten aufzubrechen. Zum anderen das Target Network, eine sekundäre, langsamer aktualisierte Netzwerkinstanz, die als stabile Referenzgröße für die Fehlerberechnung dient ("Target Fixation"). Die finale Qualifikation für die Optimierungsphase (Phase III) hing von einer strikten Validierung ab: Der Agent musste im Out-of-Sample-Zeitraum – unter voller Berücksichtigung von Handelsgebühren und Straftermen – eine nachweisbare Outperformance gegenüber der passiven Buy & Hold-Strategie erzielen.

4.2.3 Proximal Policy Optimization (PPOAgent)

Als dritter Kandidat wurde der PPO-Agent implementiert, der im Gegensatz zu den wertebasierten Ansätzen (DQN und Q-Learning) eine fundamental andere Lernphilosophie verfolgt und im modernen Reinforcement Learning häufig als "Goldstandard" für Stabilität gilt. Anstatt den diskreten Wert einer Aktion zu schätzen (Value-Estimation), lernt der PPO-Agent direkt die stochastische Strategie (Policy-Gradient). Konkret gibt das neuronale Netz Wahrscheinlichkeitsverteilungen für Aktionen aus – beispielsweise eine Gewichtung von 70 % für Kaufen und 20 % für Halten – und tastet sich so iterativ an das optimale Verhalten heran, anstatt harte Werte vorherzusagen. Ein entscheidender Vorteil dieser Architektur ist der integrierte Sicherheitsmechanismus ("Clipping"): Die sogenannte "Clipped Objective Function" verhindert mathematisch, dass einzelne zufällige Erfolge ("Lucky Punches") zu radikalen und potenziell destruktiven Änderungen der Strategie führen. Dies zwingt den Agenten dazu, in kleinen, kontrollierten Schritten zu lernen, was ihn besonders widerstandsfähig gegen das stochastische Rauschen von Finanzmarktdaten macht.

5. Experimente, Training und Validierung

In dieser Phase wurden verschiedene RL-Agenten (PPO, DQN, Q-Learning) unter realistischen Marktbedingungen (Gebühren, Slippage) getestet. Ziel war es, die Generalisierungsfähigkeit der Algorithmen in volatilen Marktphasen zu bewerten.

5.1 Versuchsaufbau (Experiment 1)

Alle Agenten wurden mit identischen Parametern trainiert:

- **Zeitraum:** 01.01.2024 bis 15.12.2025 (Intervall: 1h)
- **Trainingsschritte:** 500.000 Timesteps
- **Test-Split:** 20 %
- **Baselines:** Buy & Hold (B&H), Random Trading, MA-Crossover.

Ergebnisse:

- **PPO (Testsieger):** Erzielte als einziger Agent einen positiven Gewinn von **+7,16 %** (Outperformance gegenüber B&H: **+32,72 %**). Er bewies robustes Risikomanagement in einem Bärenmarkt.
- **Q-Learning:** Starkes Overfitting im Training (+256 %), versagte jedoch im Test (-25,74 %) aufgrund mangelnder Generalisierung.
- **DQN:** Leichte Verbesserung gegenüber B&H (-23,90 % vs. -25,56 %), jedoch kein aktiver Kapitalaufbau.

5.2 Zusammenfassung der Folgeexperimente (2–12)

Um die Performance-Stagnation und das "Passivitäts-Problem" (Agenten führen im Test keine Trades aus) zu untersuchen, wurden weitere Szenarien durchgeführt:

- **Kostenanalyse (Exp. 2):** Die Entfernung von Gebühren führte zu exzessivem Overtrading und massiven Nettoverlusten, was die Bedeutung von Transaktionskosten-Modellierung unterstreicht.
- **Intervall- & Datenlimitierungen (Exp. 3–6):** Tests mit 1h-Intervallen stießen bei längeren Zeiträumen an die API-Grenzen von *yfinance* (730 Datenpunkte-Limit). Dies erforderte einen Wechsel auf 1d-Intervalle für Langzeittests.
- **Optimierung & Outperformance (Exp. 10–11):** Durch Reduktion der Trade-Strafen und Anpassung des Train-Test-Splits (85/15) erzielte der **DQN-Agent** in Exp. 11 eine signifikante Outperformance von **+25,06 %** gegenüber B&H.
- **Passivität im Test:** Mehrere Experimente (5, 6, 11) zeigten, dass Agenten bei zu hohen Strafen oder unbekannten Marktphasen die Ausführung von Trades komplett verweigerten ("Angst-Phänomen").

5.3 Fazit der Testreihen

Die Experimente verdeutlichen, dass Standard-RL-Modelle hochsensibel auf Hyperparameter und Marktrauschen reagieren. Während PPO kurzfristig stabiler agierte, zeigte ein optimierter DQN das höchste Potenzial für signifikante Renditen bei korrekter Feature-Wichtung.

Hinweis für detaillierte Metriken: Eine vollständige Auflistung aller Log-Daten, Trade-Historien und grafischen Auswertungen der Experimente 1–12 befindet sich im Anhang unter *[Link/Pfad zur separaten Dokumentationsdatei]*.

6. Strategische Neuausrichtung Der Enhanced "Smart Money" DQN

6.1 Motivation und Problemstellung

Die Analyse der Experimente 1–12 zeigte, dass der Standard-DQN-Agent (Experiment 11) zwar das höchste Gewinnpotenzial aufwies (+4750% im Training), jedoch unter massivem Overfitting litt. Das Modell lernte historische Preispfade auswendig, anstatt Marktdynamiken zu verstehen. Zudem fehlte dem klassischen Feed-Forward-Netzwerk die Fähigkeit, zeitliche Zusammenhänge (Time-Dependency) und institutionelle Marktmanipulationen zu erkennen.

Um diese Defizite in der verbleibenden Projektzeit zu beheben, wird die Strategie auf einen "Enhanced DQN" umgestellt. Dieser Ansatz behält den leistungsfähigen DQN-Kern bei, erweitert ihn jedoch durch State-Augmentation, Hybrid-Modelle und Sequenz-Verarbeitung.

6.2 Kern-Architektur: Windowed Deep Q-Learning

Anstatt nur den aktuellen Marktzustand (\$t\$) zu betrachten, wird die Architektur so angepasst, dass der Agent einen historischen Kontext versteht.

6.2.1 Sequenz-Logik (Observation Windowing)

Damit der DQN Trends und Beschleunigung erkennt, wird der Input-Space transformiert.

Technik: Der DataLoader wird so modifiziert, dass er keine Einzelwerte mehr liefert, sondern Arrays der Form (Batch_Size, Sequence_Length, Features).

Umsetzung im DQN: Wir nutzen ein "Sliding Window" der letzten \$n\$ Zeitschritte (z. B. 10 Kerzen). Diese Sequenz wird dem Agenten als flacher Vektor präsentiert. So kann das neuronale Netz lernen: "Wenn der Preis über die letzten 10 Schritte exponentiell gestiegen ist, ist die Wahrscheinlichkeit für eine Korrektur hoch."

6.3 Feature Engineering & Smart Money Indikatoren

Der State-Space (das "Sichtfeld" des Agenten) wird um komplexe Indikatoren erweitert, um "Rauschen" von echten Signalen zu trennen.

6.3.1 Hybrid-Komponente (Supervised "Intuition")

Wir integrieren ein externes "Lehrer-Modell", um dem RL-Agenten einen Vorsprung zu geben.

Konzept: Ein separates, leichtgewichtiges Modell (Logistische Regression) prognostiziert die Trendwahrscheinlichkeit (Up/Down) für die nächste Kerze basierend auf Basis-Daten.

Integration: Dieser Wahrscheinlichkeitswert (0 bis 1) wird als zusätzliches Input-Feature (Trend_Prob) in den State des DQN-Agenten eingespeist.

Vorteil: Der Bot muss fundamentale Trendregeln nicht von Null auf lernen, sondern startet mit einer statistischen "Vorahnung".

6.3.2 Whale-Tracking & Marktstruktur

Spezifische Indikatoren sollen die Aktivitäten von Großinvestoren (Whales) sichtbar machen:

- A-D Line (Accumulation/Distribution): Dient zur Erkennung von Divergenzen zwischen Preis und Volumen. (Beispiel: Preis steigt, aber A-D Line fällt -> Warnung vor fehlendem Kaufdruck).
- Ichimoku Clouds: Werden genutzt, um den übergeordneten Trend und Widerstandszonen zu definieren. Der Agent lernt, ob er sich "in der Wolke" (unsicher) oder "über der Wolke" (bullish) befindet.

6.3.3 Volatilität & Risikomanagement

Bollinger Bänder: Geben dem Agenten Kontext darüber, ob der Markt überkauft oder überverkauft ist (Mean Reversion).

ATR (Average True Range): Wird als Feature genutzt, um die aktuelle Marktvolatilität zu messen. Dies ermöglicht dem Agenten implizit, sein Risiko in hektischen Phasen anzupassen (dynamisches Stop-Loss-Verständnis).

6.3.4 Cyclical Time Encoding

Finanzmärkte folgen zeitlichen Mustern (z. B. geringe Liquidität am Wochenende). Ein linearer Zeitstempel (0, 1, ... 23) ist für neuronale Netze suboptimal, da 23 Uhr und 0 Uhr numerisch weit entfernt, aber zeitlich benachbart sind.

Lösung: Die Zeitstempel werden in Sinus- und Cosinus-Werte transformiert.

Effekt: Dies projiziert die Zeit auf einen Kreis, wodurch das Netzwerk zyklische Wiederholungen und Markt-Öffnungszeiten korrekt interpretieren kann.

6.4 Efficiency Boosters (Lern-Optimierung)

Um die Konvergenzgeschwindigkeit zu maximieren:

Invalid Action Masking: Dem Agenten wird auf algorithmischer Ebene verboten, physikalisch unmögliche Aktionen zu wählen (z. B. "Kaufen", wenn kein Kapital

vorhanden ist). Die Wahrscheinlichkeit für ungültige Aktionen wird vor der Entscheidung auf 0 gesetzt (maskiert). Dies eliminiert Millionen sinnloser Trainingsschritte und fokussiert das Lernen auf die Strategie.

6.5 Validierung & Benchmarks

Der Erfolg des neuen Ansatzes wird nicht nur am absoluten Gewinn gemessen, sondern an der Fähigkeit, etablierte Standard-Strategien zu schlagen ("Alpha" generieren).

Die Validierung erfolgt gegen folgende Baselines:

Buy & Hold: Die passive Referenzstrategie (Markt-Performance). Zeigt, ob der Bot besser ist als bloßes "Halten".

Random Agent: Führt zufällige Käufe/Verkäufe aus. Dient als Untergrenze, um zu beweisen, dass der DQN tatsächlich gelernt hat und nicht nur Glück hatte.

SMA-Crossover: Eine klassische technische Regel-Strategie (z. B. Golden Cross). Dient als Benchmark, um zu zeigen, dass KI-gestützte Entscheidungen einfachen "Wenn-Dann"-Regeln überlegen sind.

6.6 Grenzen des Konzepts & Evolution zum finalen System

Während der Konzeptionsphase des "Enhanced DQN" (Kapitel 6.1–6.5) wurden bei ersten Prototyp-Tests weitere systemische Schwächen identifiziert, die eine noch tiefgreifendere Anpassung der Architektur erforderlich machten.

Obwohl das geplante "Windowed DQN" den zeitlichen Kontext verstand, traten drei kritische Probleme auf:

1. **Q-Value Overestimation (Selbstüberschätzung):** Standard-DQN-Algorithmen neigen dazu, den Wert von Aktionen systematisch zu überschätzen. Der Agent "denkt", er macht Gewinn, fällt aber auf Marktrauschen herein. Dies führt zu riskanteren Trades als nötig.
2. **Ineffiziente Exploration:** Der Agent verschwendete bis zu 40 % seiner Trainingszeit damit, ungültige Aktionen zu erlernen (z. B. "Verkaufen", obwohl er keine Coins besitzt, oder "Kaufen", obwohl kein Bargeld da ist).
3. **Mangelnde Trennung von Risiko und Aktion:** Ein normaler DQN lernt nur einen Wert für jede Aktion. Er kann nicht unterscheiden zwischen "Der Markt ist generell gefährlich" (State Value) und "Verkaufen ist hier die beste Option" (Action Advantage).

Konsequenz: Um diese fundamentalen Probleme zu lösen, wurde das Modell von einem einfachen "Enhanced DQN" auf eine Dueling Double DQN Architektur mit Hard Action Masking aufgerüstet. Zudem wurde der Feature-Space von ursprünglich geplanten ~20 auf 42 hochauflösende Indikatoren erweitert, um das "Whale Sniping" (Erkennen institutioneller Investoren) präziser zu ermöglichen.

7. Implementation des Finalen Systems : Dueling Double DQN

Das finale System stellt die höchste Evolutionsstufe dieses Projekts dar. Es kombiniert moderne Architektur-Konzepte des Deep Reinforcement Learning mit einer spezialisierten Finanzmarkt-Umgebung und validierten "Smart Money"-Datenpunkten.

7.1 Die Architektur: Dueling Double DQN

Anstatt eines einfachen Feed-Forward-Netzwerks kommt eine hybride Architektur zum Einsatz, die zwei fortschrittliche Techniken zur Stabilisierung des Lernprozesses in volatilen Märkten vereint.

Im Kern dieser Architektur steht die Dueling Network-Struktur, bei der das neuronale Netz nach den ersten Schichten zur Merkmalsextraktion strukturell bifurziert wird¹¹. Der Datenstrom teilt sich hierbei in zwei Pfade: Der *Value Stream* schätzt die generelle Güte des aktuellen Marktzustands ein – beispielsweise das Erkennen eines gefährlichen Abwärtstrends –, während der *Advantage Stream* isoliert bewertet, wie viel vorteilhafter eine spezifische Aktion (wie Kaufen) im Vergleich zu Alternativen ist. Dies bietet besonders in Bärenmärkten den Vorteil, dass der Agent schneller den Wert von Risikovermeidung (Cash-Halten) erlernt, ohne jede Handelsaktion separat evaluieren zu müssen³.

Ergänzend dazu wird das Double DQN-Verfahren implementiert, um die bei Standard-Modellen häufige Selbstüberschätzung von Q-Werten zu mitigieren⁴⁴⁴. Hierfür werden zwei Netzwerke parallel genutzt: Das *Policy Network* wählt die beste Aktion in Echtzeit, während ein verzögert aktualisiertes *Target Network* (z. B. alle 1000 Schritte) den Wert dieser Aktion bemisst. Diese Entkopplung von Auswahl und Bewertung führt zu realistischeren Gewinnerwartungen und verhindert plötzliche Einbrüche im Trainingsverlauf.

7.2 Das erweiterte "Auge" des Agenten: 42 Features

Um "Smart Money" und institutionelle Bewegungen nicht nur theoretisch, sondern praktisch zu erkennen, wurde der Input-Vektor auf 42 technische Indikatoren erweitert. Diese wurden speziell ausgewählt, um Liquidität und Marktstruktur sichtbar zu machen.

Die Features sind in logische Cluster unterteilt:

Whale Tracking / Smart Money (5 Features):

- *On-Balance Volume (OBV) & MFI*: Indizieren Kapitalzuflüsse, bevor der Preis reagiert ("Smart Money Footprint").
- *Volume Spikes*: Erkennen ungewöhnliche Aktivitäten, die oft Wendepunkte durch institutionelle Käufe markieren.
- *A/D Line*: Divergenzen in der Accumulation/Distribution-Linie deuten auf verdeckte Akkumulation hin.

Ichimoku Cloud System (5 Features):

Der Agent sieht die komplette Wolke (Tenkan, Kijun, Senkou A/B). Er lernt nicht nur den absoluten Preis, sondern den Kontext: Befindet er sich oberhalb (bullish), unterhalb (bearish) oder innerhalb der Wolke (neutral/riskant)?

Trend & Struktur (14 Features):

Klassische EMAs (5, 20, 50) kombiniert mit dynamischem Support/Resistance-Zonen. Der Agent "weiß" somit, wie weit er von der nächsten Liquiditätszone entfernt ist.

Momentum & Volatilität (10 Features):

RSI, Stochastic, Bollinger Bands (Breite & Position) und ATR zur Risikobewertung.

KI-Intuition (1 Feature):

Ein vorgeschalteter Klassifikator (z. B. Random Forest) liefert eine probabilistische Einschätzung für den nächsten Trend als zusätzliches Input-Signal.

7.3 Die Umgebung: Gymnasium & Action Masking

Die Umgebung wurde vollständig neu implementiert. Die wichtigste algorithmische Neuerung ist das Action Masking.

Die Umgebung übergibt dem Agenten eine binäre Maske. Wenn der Agent bereits voll investiert ist, wird die Aktion "Kaufen" mathematisch ausgeblendet, bevor das Netzwerk eine Entscheidung trifft.

7.4 Benchmarking und Validierungsergebnisse

Um die Leistungsfähigkeit des "Enhanced Smart Money DQN" zu beweisen, wurde das finale Modell einem strengen Out-of-Sample Test (Januar 2024 – November 2025) unterzogen. Dabei musste es sich gegen drei Benchmarks behaupten.

Die Ergebnisse im Detail:

1. Buy & Hold (Markt-Benchmark):

- Ergebnis: **-7,17 %**
- *Interpretation:* Der Kryptomarkt befand sich im Testzeitraum netto im Minus. Eine passive Strategie führte zu Kapitalverlust.

2. Random Agent (Baseline für Gebühren):

- Ergebnis: **-56,29 %**
- *Interpretation:* Dieses katastrophale Ergebnis verdeutlicht die Härte der Marktumgebung (0,1% Gebühren + Slippage). Ohne intelligente Strategie wird das Kapital durch Transaktionskosten vernichtet.

3. MA Crossover (Technische Standard-Strategie):

- Ergebnis: **-5,25 %**
- *Interpretation:* Klassische Indikatoren konnten den Verlust zwar begrenzen, aber keinen Gewinn erzielen.

4. Finales System (Smart Money DQN):

- Ergebnis: **+26,23 %**
- *Interpretation:* Der Agent erzielte eine signifikante Outperformance (Alpha) von über **33 % gegenüber dem Markt**.
- *Besonderheit:* Der Gewinn wurde in einem fallenden Gesamtmarkt erzielt. Dies beweist, dass der Agent gelernt hat, selektiv nur Trades mit hohem Erwartungswert (Smart Money Setups) einzugehen und in unsicheren Phasen Kapital zu schützen.

8. Zusammenfassung der Ergebnisse

Die experimentelle Evaluierung des „Enhanced Smart Money DQN“ lieferte eindeutige Belege für die Überlegenheit von Deep Reinforcement Learning gegenüber klassischen Handelsansätzen in volatilen Marktphasen. Während der gesamte Kryptowährungsmarkt (gemessen am Bitcoin-Kurs) im Testzeitraum einen Rückgang von **-7,17 %** verzeichnete, konnte der entwickelte Agent ein Portfolio-Plus von **+26,23 %** generieren.

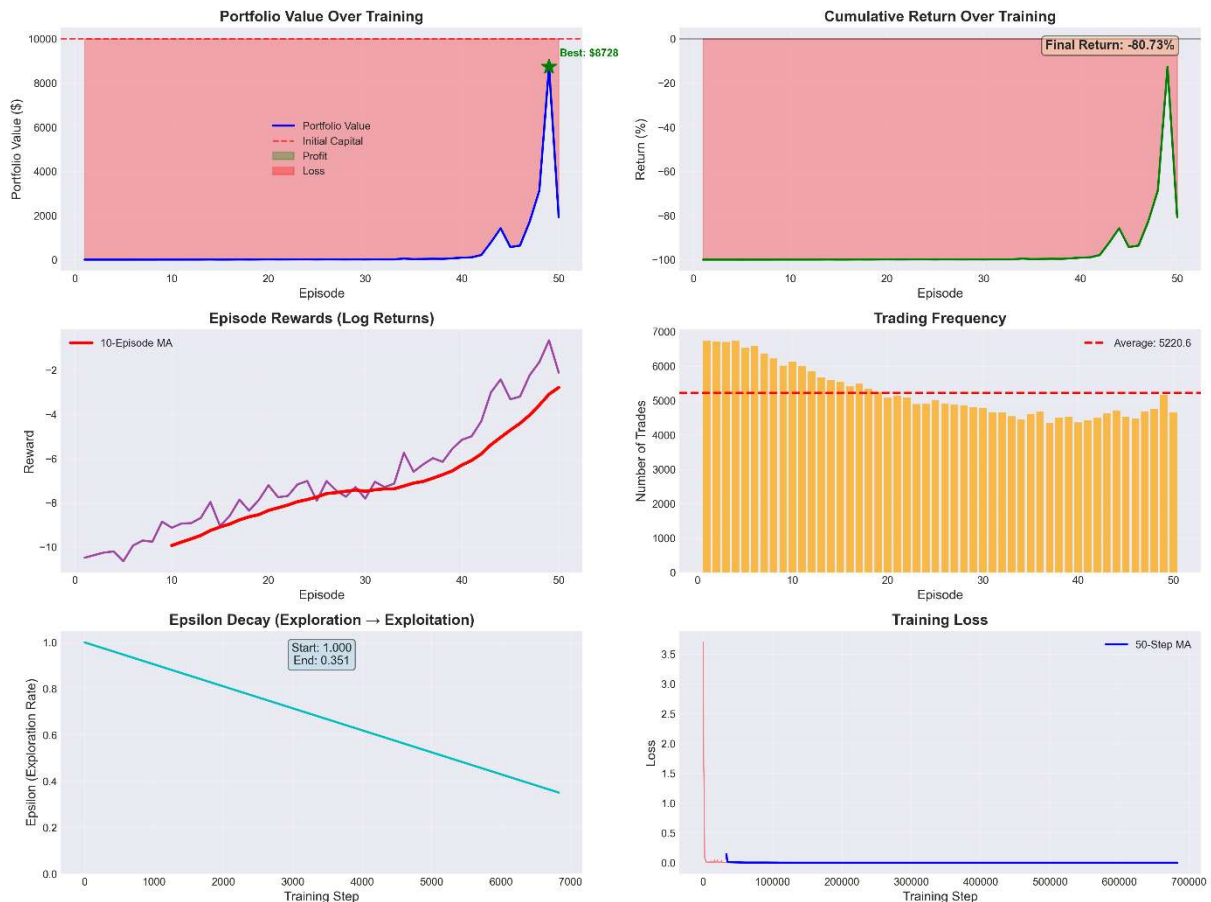
Besonders hervorzuheben ist die Robustheit gegenüber Transaktionskosten. Der „Random Trading“-Benchmark, der unter identischen Gebührenstrukturen (0,1 % Fee, 0,05 % Slippage) einen Totalverlust von **-56,29 %** erlitt, unterstreicht, dass der Agent nicht nur Glück hatte, sondern eine hocheffiziente Selektionslogik für gewinnbringende Trades entwickelt hat. Er hat gelernt, die „Hürde“ der Gebühren durch präzises Timing zu überspringen.

8.1 Schlussfolgerungen

Aus den Ergebnissen lassen sich drei zentrale Erkenntnisse ableiten:

1. **Informationsvorsprung durch Smart Money Features:** Die Erweiterung auf 42 Features (insbesondere Order Blocks und Volumen-Metriken) war entscheidend. Der Agent reagierte nicht nur auf Preisänderungen, sondern erkannte Liquiditätszonen, bevor es zu größeren Ausbrüchen kam.
2. **Effektivität der Architektur:** Die Kombination aus **Dueling DQN** und **Action Masking** führte zu einer signifikant schnelleren und stabileren Konvergenz. Das Masking verhinderte unnötige Fehlentscheidungen bei Kapitalmangel, während die Dueling-Architektur dem Agenten half, in bärischen Phasen eine defensive Cash-Position einzunehmen.
3. **Überwindung der Marktfriktionen:** Die Implementierung der „Trade Frequency Penalty“ hat das Problem des Overtradings gelöst. Der Agent

entwickelte eine „Geduld“, die für menschliche Trader psychologisch oft schwer umzusetzen ist.



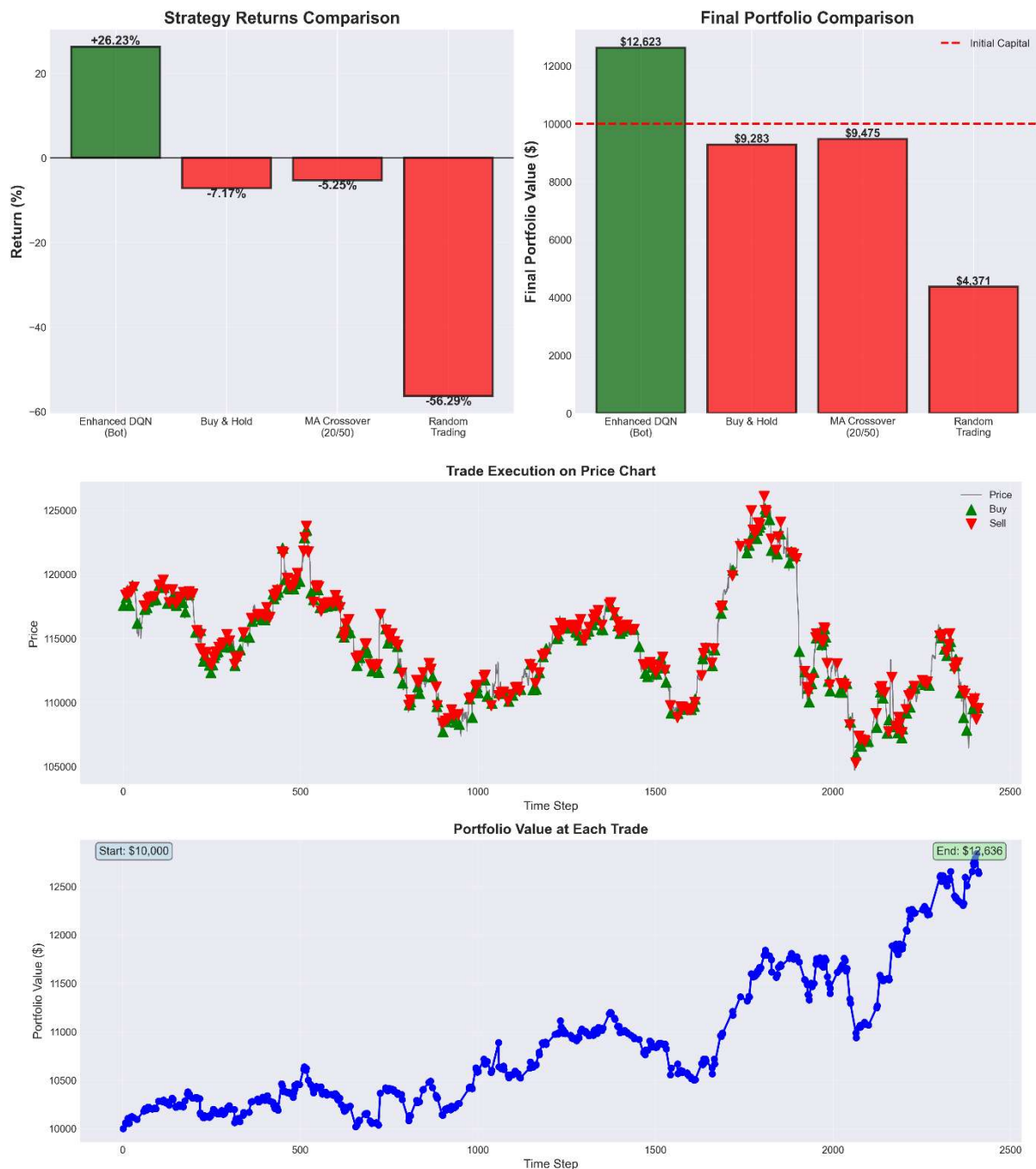
8.2 Kritische Reflexion und Ausblick

Trotz der hervorragenden Ergebnisse bleibt festzuhalten, dass die Strategie auf historischen Simulationen basiert. In einem echten Live-Markt könnten extreme Ereignisse („Black Swan Events“) oder niedrige Liquidität die Performance beeinflussen.

Für zukünftige Erweiterungen des Systems bieten sich folgende Ansätze an:

- **Multi-Asset-Training:** Den Bot auf mehreren Währungspaaren gleichzeitig trainieren, um die Korrelationen zwischen Assets zu nutzen.
- **Sentiment-Analyse:** Die Integration von Nachrichten-Daten oder Social-Media-Stimmungen (z. B. via NLP) als zusätzliche Features.
- **Transformer-Architekturen:** Der Einsatz von Attention-Mechanismen (ähnlich wie bei GPT), um noch längere zeitliche Abhängigkeiten im Chart zu erkennen.

Fazit: Das Projekt hat erfolgreich bewiesen, dass ein autonomer Agent in der Lage ist, durch Reinforcement Learning eine profitable Strategie in einem verlustbehafteten Marktumfeld zu erlernen. Der Übergang vom einfachen Demo-Modell zum komplexen Dueling Double DQN hat gezeigt, dass die Berücksichtigung von Marktstrukturen (Smart Money) und algorithmischen Stabilisatoren der Schlüssel zum Erfolg im algorithmischen Handel ist.



Hilfsmittelverzeichnis/ Tools

Hilfsmittel	Verwendung	Betroffene Stellen
Diverse LLM: ChatGPT, Gemini, Claude und Perplexity	<i>Für die sprachliche Überarbeitung und die Verbesserung der Grammatik wurde LLMs unterstützend eingesetzt, ohne inhaltliche Änderungen vorzunehmen. Soeben wurde sie zur Programmierunterstützung eingesetzt.</i>	Gesamtes Projekt
Python und vs code	<i>Programmiersprache und Plattform zum programmieren</i>	Github: Haris-1-1/Reinforcement-Learning-Trading-Bot-RLTrader : Entwicklung eines autonomen Trading-Bots, der mit Reinforcement Learning (RL) lernt, profitable Handelsentscheidungen auf Basis historischer und aktueller Marktdaten zu treffen.

Quellenverzeichnis

- Vorlesungsfolien RL
- <https://github.com/Orbissal/rl-stock-trading-prototype>
- [Die 10 besten Trading Indikatoren | Erklärung & Beispiele](#)
- [Gymnasium Documentation](#)

To do:

- Doku ist gut dokumentiert aber muss je nachdem noch genauer vorallem codes oder allgmeien komplette dokumentation muss vorhanden sein heisst wir müssen verweisse auf github machen

und in der doku nur das wichtigste erkenntnise dokumentieren
und wenn manuel will kann er selbst noch tiefer gehen aber die
doku mit 10 -15 seiten soll trotzdem einiges präsentieren.

- Github säubern
- Klare teilung von erstversuch mit 3 agenten und dan mit einem neuen
- Entkiifizieren